

A parallel algorithm for network traffic anomaly detection based on Isolation Forest

Xiaoling Tao^{1,2,3}, Yang Peng¹ , Feng Zhao¹, Peichao Zhao¹ and Yong Wang¹

Abstract

With the rapid development of large-scale complex networks and proliferation of various social network applications, the amount of network traffic data generated is increasing tremendously, and efficient anomaly detection on those massive network traffic data is crucial to many network applications, such as malware detection, load balancing, network intrusion detection. Although there are many methods around for network traffic anomaly detection, they are all designed for single machine, failing to deal with the case that the network traffic data are so large that it is prohibitive for a single computer to store and process the data. To solve these problems, we propose a parallel algorithm based on Isolation Forest and Spark for network traffic anomaly detection. We combine the advantages of Isolation Forest algorithm in network traffic anomaly detection and big data processing capability of Spark technology. Meanwhile, we apply the idea of parallelization to the process of modeling and evaluation. In the calculation process, by assigning tasks to multiple compute nodes, Isolation Forest and Spark can efficiently perform anomaly detection and evaluation process. By this way, we can also solve the problem of computation bottleneck on single machine. Extensive experiments on real world datasets show that our Isolation Forest and Spark is efficient and scales well for anomaly detection on large network traffic data.

Keywords

Network traffic, anomaly detection, Isolation Forest, Spark, parallelization

Date received: 2 August 2018; accepted: 16 October 2018

Handling Editor: Wei Li

Introduction

Due to the development of new applications such as social networks, location based service, video sharing, the scale of Internet continues to expand. At the same time, network traffic data are also showing a trend of explosive growth, which has brought severe challenges to network traffic anomaly detection. Specially, the security and privacy problems caused by these data and their solutions are particularly important.^{1,2} There are many kinds of definitions of anomaly, one widely accepted definition is that of Hawkins:³ “An anomaly is an observation which deviates so much from other observations as to arouse suspicions that it was

¹Guangxi Colleges and Universities Key Laboratory of Cloud Computing and Complex Systems, Guilin University of Electronic Technology, Guilin, China

²State Key Laboratory of Integrated Service Networks (ISN), Xidian University, Xi'an, China

³Guangxi Cooperative Innovation Center of Cloud Computing and Big Data, Guilin University of Electronic Technology, Guilin, China

Corresponding author:

Feng Zhao, Guangxi Colleges and Universities Key Laboratory of Cloud Computing and Complex Systems, Guilin University of Electronic Technology, Guilin 541004, China.
Email: zhaofeng@guet.edu.cn



generated by a different mechanism.” To alleviate the impact of abnormal network traffic, network anomaly detection plays a key role on anomaly handling. Network anomaly detection was originally proposed by Denning,⁴ which refers to filtering out abnormal information from traffic data, identifying and diagnosing the security status of the network so as to ensure proper functioning of the network.

Network anomaly detection is a crucial part of network security; it is an useful technology to understand the status and performance of network, which is important to the management of Internet and systems. Network anomaly detection has been applied to many fields, such as wireless sensor networks (WSNs),⁵ mobile network,⁶ healthy and medical application.^{7,8}

The current mainstream methods for detecting anomalies from network traffic include statistical based, time series-based, sketch-based, and machine learning-based methods. Among them, the Isolation Forest algorithm has many advantages, including high accuracy at low false positive rates, high efficiency and extensibility, compared to others. In particular, its computational complexity is linear with respect to the size of data sample, and it can deal with high-dimensional data. However, the Isolation Forest algorithm is designed for single computer, meaning that it cannot work when the size of dataset exceeds the memory limit of a single machine. Therefore, in order to process large-scale network traffic data, it is necessary to realize the parallelization of the Isolation Forest algorithm.

To solve the above problems, we propose a parallel algorithm for network traffic anomaly detection based on Isolation Forest and Spark (SPIF). Our contributions in this work can be summarized as follows:

1. We use parallel strategies to construct multiple trees simultaneously, which improves the efficiency of modeling process.
2. We propose a parallel algorithm for network anomaly evaluation. We use parallel strategies to evaluate multiple data simultaneously, which improves the efficiency of abnormal evaluation.
3. We integrate our algorithm in the Spark framework. The data read operation of Spark is based on memory's secondary caching mechanism, so it can avoid frequent disk Input/Output operations and thus improve the data processing efficiency.

The remainder of this article is organized as follows. Section “Related work” reviews the related work. In section “Preliminaries,” we provide a brief introduction to some preliminaries. Then, we propose our parallel algorithm for network traffic anomaly detection based on SPIF in section “Proposed model.” In section “Experimental study,” we briefly introduce the

experimental environment and data we used, and the experimental results are analyzed in detail. Finally, conclusions are drawn and further work is given in section “Conclusion.”

Related work

Network traffic anomaly detection is usually classified into four main categories: statistical based, time series based, sketch based, and machine learning based. For sketch based and machine learning based approaches, the detection system needs labeled data to train a detection model which is quite time consumption. However, they always have a higher accuracy. As for the first two categories, they are mostly unsupervised, which means they do not need labeled data.

Statistical-based approaches are suitable for the detection of anomaly. Matsuda et al.⁹ used principal component analysis (PCA)-based network traffic anomaly detection technology to project the measured flow data into normal subspace and abnormal subspace to detect traffic anomaly. This method increased the robustness of anomaly detection system and reduced the computational cost. De la Hoz et al.¹⁰ proposed a network traffic anomaly detection method based on PCA and self-organizing map (SOM), which can quickly realize the intrusion detection system (IDS) to deal with the current link bandwidth. Berezinski et al.¹¹ studied the application of anomaly detection in the field of network intrusion detection and verified that the entropy-based method is suitable for the abnormal detection of modern botnet. In addition, Markov model was also anomaly traffic detection methods which are based on statistical analysis.¹² Markov anomaly detection model has self-learning function, it can determine training time according to data size, and it is easy to implement. But its real-time performance of anomaly detection is poor, and it is difficult to detect abnormal in a relatively short time.

Time series-based approaches include auto-regression and moving average model (ARMA) regression,¹³ wavelet transform,¹⁴ empirical mode decomposition (EMD) transformation, instantaneous frequency analysis, and so on. When these methods are used for network traffic anomaly detection, they are suitable to deal with the network traffic data that meet the requirements of quantization, and they can use the technology of signal processing flexibly. The research of Celenk et al.¹⁵ presented a method of anomaly detection based on Wiener filtering of noise and ARMA modeling of network flow data. In the research, they use network-monitoring metrics for traffic features to dynamically calculate noise and traffic signal statistics. Han and Zhang¹⁶ performed a real-time EMD on the network traffic and used weighted self-similarity parameter to detect abnormal activities over the Internet. Yu et al.¹⁷

proposed a traffic anomaly detection algorithm for WSNs based on the autoregressive integrated moving average (ARIMA) model. Jiang et al.¹⁸ proposed a high-speed backbone network traffic anomaly detection method based on multi-scale analysis. In the first step of this method, network traffic will be transformed on a continuous scale, and the PCA will be carried out. Then, they can extract the characteristics of abnormal network traffic and construct new mapping functions to detect abnormal traffic.

Sketch is a distributed profile data structure that can handle large amounts of data in a short time, so it is widely used in network traffic anomaly detection. Huang and Lee¹⁹ proposed a data structure of traffic anomaly detection based on distributed architecture. This structure combines the traditional counter-based and sketch-based technologies; its detection phase is divided into two stages: local detection and distributed detection, which ensures the accuracy and scalability of the model. Chen et al.²⁰ implemented the detection of abnormal Internet Protocol (IP) source address by combining the sketch data structure with the improved multi-scale principal component analysis (MSPCA) detection algorithm. This approach takes advantages of the characteristics of PCA and wavelet analysis, so MSPCA can identify anomalies efficiently.

Machine learning-based approaches can quickly and effectively deal with large-scale network traffic data through self-learning method. The main machine learning methods include classification, clustering, pattern recognition, neural network, and decision tree. Literature^{21,22} applied the clustering of machine learning to network traffic anomaly detection and improved the efficiency of anomaly detection. Shon et al.²³ proposed a machine learning framework for anomaly detection which uses genetic algorithm (GA) for feature selection and support vector machine (SVM) for packet classification. Casas et al.²⁴ solved the automation problem of network traffic anomaly detection and classification using decision tree and proposed a multi-detector method to improve the performance of the whole anomaly detection system. Sheikhan and Jadidi²⁵ used multi-layer perceptron (MLP) neural classifier to distinguish benign and malicious flow of network intrusion detection system (NIDS) based on flow, and on this basis, they used modified gravitational search algorithm (MGSA) to optimize the interconnection weight of the neural anomaly detector and improved the performance of the detection system.

Preliminaries

Isolation Forest

There are two quantitative characteristics used in our proposed method: (1) anomalies account for a very

small proportion of the whole dataset and (2) there is a big difference in their attribute values between the anomaly and normal instance, which make them “few and different” and more susceptible to isolate them from normal instances. In this article, we use tree structure to identify every single instance. Since anomalies’ susceptibility to isolation, they will be closer to the root of the tree. On the contrary, normal instances will locate at the leaf level of the tree. Those are the foundations of our method for anomaly detection, and we call it Isolation Tree or iTree.²⁶

Isolation Forest or iForest is a combination of iTrees, which is an efficient method of anomaly detection based on ensemble, and it is an effective and popular algorithm that can deal with large data. It is different from traditional anomaly detection methods for its high accuracy, linear time complexity, and low memory cost.

Isolation Forest returns the anomaly score of each sample using the Isolation Forest algorithm. The Isolation Forest “isolates” observations by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature. Since recursive partitioning can be represented by a tree structure, the number of splittings required to isolate a sample is equivalent to the path length from the root node to the terminating node. This path length, averaged over a forest of such random trees, is a measure of normality and our decision function. Random partitioning produces noticeably shorter paths for anomalies. Hence, when a forest of random trees collectively produces shorter path lengths for particular samples, they are highly likely to be anomalies.

In our work, iForest algorithm only has two parameters: the member of iTrees n and the size of data sample of training dataset m . Normally, the coverage of iTrees’ path length is comprehensive when n is 100. Since sampling can better separate abnormal data from normal data, so the size m is important to the performance of the algorithm. However, the ability of iForest algorithm to detect anomalies will degenerate if the value of m is set too high, and empirically $m = 256$ will be appropriate.

The iForest algorithm contains two stages: (1) modeling stage, which randomly samples the dataset to get subsets and builds an ensemble of iTrees; (2) evaluating stage, which passes test data through iTrees and records the path length of each test instance, then calculates the anomaly score for each test instance. The key of iForest algorithm is the construction of Isolation Forest, which is a collection of iTrees and each iTree is a binary tree.

Modeling stage. In this stage, iTrees will be constructed by recursively executing the process of partitioning of the giving dataset until instances are divided up

completely or the tree reaches the maximum depth. Details of this stages are given as follows:

1. Randomly collect m instances from dataset D as a data sample d and put it in the root node.
2. Randomly select an attribute and a split point p from d , and the split point should be generated between the maximum and minimum values of the specified attribute in the data at the current tree node.
3. The split point p generates a hyperplane to divide the current node data space into two subspaces. The data that are less than p in the specified dimension will be placed on the left subtree of the current tree node, while the other will be placed on the right subtree.
4. Repeat steps 2 and 3 in the child nodes, and continue to construct the left and right subtrees, until any of the three conditions (1) $|d| = 1$, (2) all data in d have the same value, or (3) the tree reaches a highest limit is met.
5. Repeat the above steps 1–4, produce n iTree, and construct the iForest anomaly detection model accordingly.

Evaluating stage. Evaluating stage is the process to evaluate every instance on the basis of the iTrees constructed in modeling stage and assign a anomaly score to each instance. Details of this stages are given as follows:

1. For each instance x , let x go through each iTree in the model and finally record the location of x on each iTree.
2. Calculate the path length of instance x , and the anomaly score of x according to the formula of the anomaly score.
3. Use the abnormal score obtained from step 2 to evaluate instance x .

In order to accurately identify anomaly instance, some definitions are given below.

Definition 1. Anomaly score. Given an instance x , we define its path length $h(x)$ in a specified subtree as the number of edges that x passes from the root node to the leaf node where it is located. Anomaly score $s(x, n)$ of x is defined as

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \quad (1)$$

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \quad (2)$$

where $E(H(x))$ represents the average path length of instance x in each iTree. As $c(n)$ is the average of $h(x)$, we use it to normalize $h(x)$. And $H(k) = \ln(k) + \varepsilon$,

where ε is Euler's constant, whose value is 0.5772156649.

In equation (1)

1. When $E(h(x)) \rightarrow 0$, $s \rightarrow 1$, that means that if instances are assigned a s score very close to 1, then they are more likely to be anomalies.
2. When $E(h(x)) \rightarrow c(n)$, $s \rightarrow 0.5$, that means that if all the instances are obtained s value ≈ 0.5 , then there is no significant anomalies.
3. When $E(h(x)) \rightarrow (n-1)$, $s \rightarrow 0$, that means that if instances are assigned a s score much smaller than 0.5, then they are more likely to be identified as normal instances.

Big data processing platform

Spark is an open source and universal distributed computing framework that borrows from MapReduce's distributed concurrency ideas, and it uses memory-based methods to store intermediate results, which is more efficient than Hadoop and is more suitable for batch processing of large-scale data. Spark has the following advantages:

1. Running speed: Spark introduces the directed acyclic graph (DAG) execution engine and memory-based computation into the processing of big data, which increases performance by nearly 100 times.
2. Easy to use: Compared to Hadoop, which provides only Map and Reduce operations, Spark offers dozens of operations, and it supports three languages—Java, Python, and Scala, and can be implemented on a Hadoop cluster.
3. Versatility: Spark's ecosystem is composed of multiple components, and components can be called from each another, which provides a one-stop data processing platform for seamless integration and avoids the independent running of multiple systems. Moreover, it also allows advanced components benefit from improvements in the underlying components.
4. Run everywhere: Spark can implement the read operation of HBase, Hadoop, S3, Cassandra, and Tachyon, achieve the read–write operation of the native data of the persistence layer, and is highly adaptable.

Spark has obvious advantage in big data processing, it has been used by many researchers and industrial practitioners worldwide and has become the top project of Apache. In recent years, the ecosystem of Spark has been gradually improved and is fully functional, and the Spark ecosystem has many components, as shown in Figure 1.

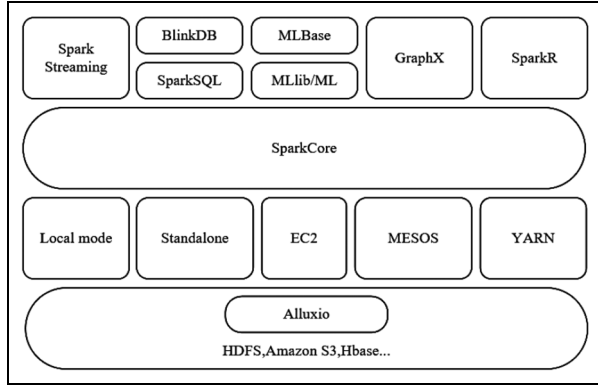


Figure 1. The ecosystem of Spark.

Spark Core is the core component of Spark ecosystem; its primary responsibility is to implement data reading and application analysis, which ensures the execution efficiency of distributed computing by taking advantages of memory computing and directed acyclic graph mechanism to minimize the data reading time for multiple iterations. Besides, Spark Streaming provides Streaming real-time big data processing services, Spark Submit and Spark Shell provide interactive services for batch processing, SparkSQL provides instant query service, Machine Learning Library (MLlib) provides Machine Learning algorithm service, GraphX provides image processing services, and SparkR provides data analysis and computing services. Multiple components can be seamlessly combined with each other, which can be applied to most of today's task scenarios.

Proposed model

Although the traditional iForest algorithm has the advantages of high accuracy, linear time complexity, and low memory requirement, it has a obvious disadvantage in dealing with big data—it is based on a single machine design, which means it is unable to handle large-scale data. With the explosive growth of network traffic data, the computational and storage limit of a single node can no longer meet the need of anomaly detection, and this greatly limits the applicability of the iForest algorithm.

To solve the problem of computation bottleneck on single machine, we propose a parallel model for network traffic anomaly detection based on Isolation Forest. The system architecture is described in Figure 2.

System model

The model is composed of network traffic data acquisition layer, network traffic data preprocessing layer, network traffic anomaly detection layer, and application service layer. Each layer is independent of the others,

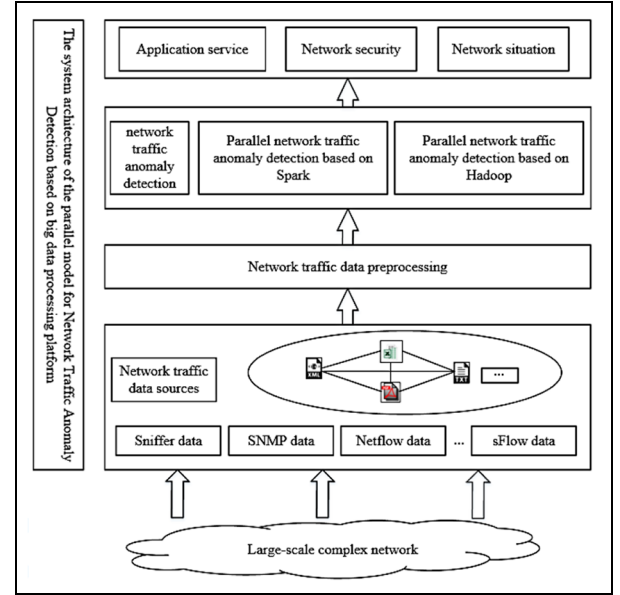


Figure 2. The system architecture of the parallel model for network traffic anomaly detection based on big data processing platform.

but they are closely linked through the big data processing platform, and they need work together to complete the processing of network traffic anomaly detection.

The network traffic data acquisition layer is one of the basic layers of the model. It uses a variety of network traffic acquisition techniques to collect large-scale network traffic of corresponding network nodes, and it is also responsible for transferring the collected network traffic information to the network traffic preprocessing layer.

Network traffic data preprocessing layer is also one of the basic layers of the model. The main task of this layer is to use the data preprocessing algorithm to process the data into the format required by the network traffic anomaly detection method. After all operations are finished, the data will be handed over to the network traffic anomaly detection layer.

The network traffic anomaly detection layer is the core layer of the model. Its first task is to ensure the parallelization of network traffic anomaly detection through the big data processing platform—Spark and Hadoop, improve the efficiency of anomaly detection, and submit the test results to the application service layer.

The application service layer can provide a series of traffic anomaly detection application services and provide the basis for the analysis of network security situation. This layer is based on the network traffic anomaly detection layer.

The big data processing platform is the carrier of the whole model, which provides powerful computation power. As a distributed computing framework for large-scale network traffic data, Spark can realize

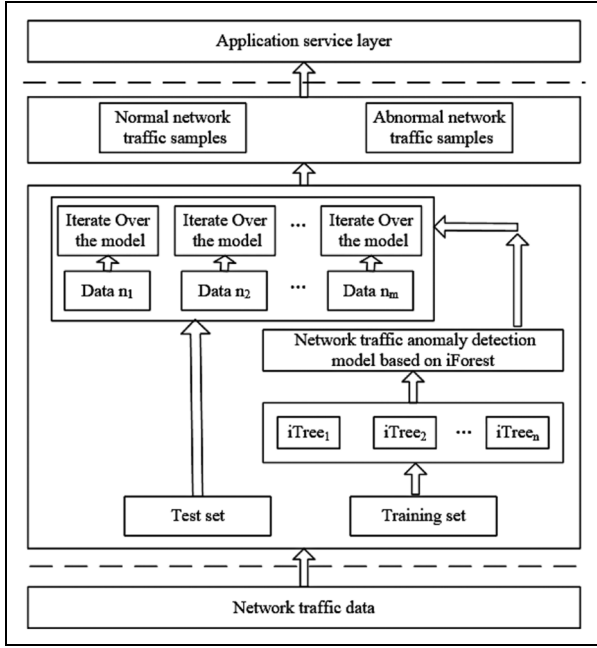


Figure 3. The system framework of SPIF.

efficient use of cloud computing resources and it is highly scalable.

Algorithm implementation

To solve the problem that iForest algorithm is unable to deal with large-scale network traffic data, we take advantages of the iForest algorithm in anomaly detection and efficiency of Spark technology in big data processing and design a parallel algorithm for anomaly detection based on SPIF. This method realizes the parallelization of the iForest algorithm in modeling process and the batch processing of anomaly evaluation. The frame of the proposed model is presented in Figure 3.

Modeling stage. Since the iForest algorithm needs to build multiple iTrees, this sequential construction method is time-consuming and is limited by the maximum memory capacity, and cannot adapt to anomaly detection on large-scale network traffic data. However, the construction process of each iTree is independent and has no influence on each other, so it is feasible to improve performance through parallel execution. In view of the above reasons, this article proposes the SPIF algorithm. Based on the process of model construction, SPIF uses the Spark platform to divide the work of iTrees construction across multiple computation nodes to execute simultaneously, thus achieving parallelism of the construction process. Algorithm 1 gives the pseudo-code of iForest for constructing iTrees in parallel.

In Algorithm 1, D represents the network traffic dataset, n represents the number of iTree, and $sampleSize$ represents the size of data sample. First, we set the maximum depth of iTree to $\log_2 sampleSize$ and initialize the iForest through Steps 1 and 2. Then, from Steps 3 to 7 of Algorithm 1, the construction of iTrees will be partitioned into multiple tasks and executed simultaneously on multiple computation nodes. In this way, parallel construction of iTrees is achieved. In the process, one data sample of instance will be picked out at random. Next, the function *buildTree()* is responsible for the construction of iTree. Finally, the iTree constructed in the previous step will be incorporated into *iForest*. When Algorithm 1 terminates, a collection of iTrees is returned, which collectively constitutes the proposed model of network traffic anomaly detection based on iForest.

In Step 4 of Algorithm 1, the *sample()* function has two parameters, that is, D and $sampleSize$. Based on a pre-specified $sampleSize$, a data sample with size $sampleSize$ will be generated, which is randomly selected from D .

Evaluation stage. The original iForest algorithm can only evaluate one instance at a time. In the process of calculating the anomaly score, the average path length of an instance needs to be calculated, and the calculation of anomaly score of each instance needs to iterate through the whole set of iTrees, which is time-consuming. To solve this problem, SPIF algorithm adopts the strategy of parallel evaluation of multiple instances at the same time, to improve the efficiency of anomaly evaluation. The process of anomaly evaluation of SPIF algorithm is given in Algorithm 2.

In Algorithm 2, D is the input dataset, iForest is the proposed model generated in Algorithm 1, and max is the threshold of anomaly score. An instance whose anomaly score is greater than max will be regarded as anomaly. On the contrary, it will be regarded as normal instance. The first step of the algorithm is to initialize the anomaly set Q , and Steps 2 to 7 are responsible for the traversal operation of instances on the iForest model and the calculation operation of anomaly score of each instance. This process is an iterative process; every instance x in D will be executed once. The *EvaluateIForest()* function is used to calculate the average path length of each instance in the iForest model, and *avgLength* is used to save the result. Next, the function *calAnomalyScore()* with four parameters: $x, label, iTrees, sampleSize$ will calculate the anomaly score of each instance and save the result in *AnomalyScore*. Then, from Steps 8 to 9, we can use the anomaly score calculated in Step 6 to isolate each instance and put the anomalies into set Q . The result of Algorithm 2 is a set of anomalies Q .

Algorithm 1. SPIF-Construction($D, n, \text{samplesize}$).

Input: D -input data, samplesize -data sample size, n -number of $i\text{Trees}$

Output: a set of n $i\text{Trees}$ ($i\text{Forest}$)
 1: Initialize $i\text{Forest}$;
 2: $\text{maxDepth} \leftarrow \lceil \log_2 \text{samplesize} \rceil$;
 3: for $i = 1$ to n do
 4: $D_i \leftarrow \text{sample}(D, \text{samplesize})$;
 5: $i\text{Tree}_i \leftarrow \text{buildTree}(D_i, \text{maxDepth})$;
 6: $i\text{Forest} \leftarrow i\text{Forest} \cup \{i\text{Tree}_i\}$;
 7: endfor
 8: return $i\text{Forest}$

Algorithm 2. SPIF-AnomalyEval($D, i\text{Forest}$).

Input: D -input data, $i\text{Forest}$ -a set of n $i\text{Tree}$, samplesize -data sample size, max-threshold of Anomaly Score

Output: Q -dataset of anomalies.
 1: Initialize Q ;
 2: for each instance $x \in D$ do
 3: for each instance $i\text{Tree} \in i\text{Trees}$ do
 4: $\text{length} \leftarrow \text{avgLength}(\text{samplesize}.getValue());$
 5: $\text{avgLength} \leftarrow \text{EvaluateForest}();$
 6: $\text{AnomalyScore} \leftarrow \text{calAnomalyScore}(x, \text{label}, i\text{Trees}, \text{samplesize});$
 7: endfor
 8: if $\text{AnomalyScore} > \text{max}$ then
 9: $Q \leftarrow Q \cup \{x\}$
 10: endif
 11: endfor
 12: return Q ;

For Algorithm 1, we suppose the time cost to construct a $i\text{Tree}$ is α and then the time complexity of Algorithm 1 to construct the n $i\text{Trees}$ is $O(n * \alpha)$. For Algorithm 2, we assume that the time spent for each $i\text{Tree}$ traversal is β and then the time complexity of Algorithm 2 is $O(|D| * n * \beta)$. Obviously, they both have linear time complexity, which fits well with the characteristics of the isolation algorithm.

Experimental study

To evaluate the effectiveness of our proposed SPIF algorithm, we conduct extensive experiments on real network traffic datasets. We investigate the performance of the proposed algorithm from three aspects, that is, detection efficiency, effectiveness, and scalability on Spark platform and Hadoop platform. To improve the reliability of experimental results, we repeat each experiment 10 times and take the average as final result.

Experimental environment

The experiment is carried out on cloud platform. The spark platform we built consists of four machines (or

Table 1. The software configuration of Spark cluster.

Categories	Version
OS	Centos6.5
Java	Java1.8.0
Hadoop	Hadoop2.6.0
Spark	Spark2.1.0

Table 2. The software configuration of Hadoop cluster.

Categories	Version
OS	Centos6.5
Java	Java1.8.0
Hadoop	Hadoop2.6.0

Table 3. Resource configuration of Spark and Hadoop cluster.

Node	Memory	CPU
Node1	8 GB	Intel(R) Xeon(R) CPU E5-2640 2.60 GHz
Node2	8 GB	Intel(R) Xeon(R) CPU E5-2640 2.60 GHz
Node3	8 GB	Intel(R) Xeon(R) CPU E5-2640 2.60 GHz
Node4	8 GB	Intel(R) Xeon(R) CPU E5-2640 2.60 GHz

nodes), where one node is configured as the master and the other three are worker nodes. The configuration of the cluster is shown in Tables 1–3.

Experiment dataset

The experiment dataset used in the experiment is UNSW-NB15^{27,28}—the comprehensive dataset used by the latest network IDS in academia. The dataset is obtained by the network security laboratory of Australia's network security center, which is created using IXIA PerfectStorm tool, and it is used to simulate the normal network activity and attack behavior of real applications. The dataset consists of four csv files, a total of 2,540,404 records, and each csv file contains attack records and normal records. In this data set, there are 300,000 anomalies, a total of 49 network traffic characteristics and 9 types of attacks. Summary of the dataset is shown in Table 4.

In order to meet the requirements of different experiments, we divide the dataset to different sizes, as shown in Table 5.

Experiment result and analysis

Detection efficiency. In order to verify the performance of the proposed SPIF method in terms of detection

Table 4. Summary of dataset UNSW-NB15.

Categories	Number
Total	2540404
Normal	2218761
Generic	215481
Exploits	44525
Fuzzers	24246
DoS	16353
Reconnaissance	13987
Analysis	2677
Backdoors	2329
Shellcode	1511
Worms	174

efficiency, this section compares SPIF with the iForest algorithm in the single-machine environment, and a parallel algorithm for anomaly detection based on Isolation Forest and Hadoop (HPIF for short). Since this experiment is to verify the anomaly detection efficiency of massive network traffic data in a single-machine environment and a cluster environment, we need large-scale data. In order to increase the reliability of the experimental results, we use datasets Data2, Data3, Data4, and Data5 to verify the efficiency of the algorithms. Figure 4 gives the running time of the iForest algorithm based on single-machine environment, HPIF and SPIF, under different settings of data size and the number of iTrees.

From Figure 4, we can see that SPIF and HPIF are significantly better than the iForest algorithm in single-machine environment when dealing with large-scale network traffic data. When dataset size and the number of iTrees are both small, the performance of the iForest algorithm based on single environment is slightly different from SPIF and HPIF. With the increase of network traffic dataset size, SPIF and HPIF obviously outperform the iForest algorithm based on single-machine environment, and the performance gap is widening as the amount of data increases. This is because SPIF algorithm and HPIF algorithm can distribute the work of modeling to the nodes of the cluster in parallel. When the number of iTrees increases, the task allocation between nodes will also be adjusted automatically. Hence, SPIF algorithm and HPIF algorithm are not sensitive to the number of iTrees. However, the iForest algorithm based on single-machine environment can

only build iTrees in a sequential manner; as the number of iTrees increases, its running time will also increase linearly. Meanwhile, SPIF algorithm is based on Spark, which can put network traffic data in the memory cache, so it can read data directly from memory when iterative operations are executed. This can avoid excessive disk I/O operations, improve the iterative efficiency, and greatly reduce processing time.

Experimental results show that when dealing with anomaly detection tasks on large-scale network traffic data, the proposed SPIF method clearly outperforms HPIF algorithm and the iForest algorithm based on single-machine environment, since SPIF reduces the running time for anomaly detection.

Effectiveness. We use UNSW-NB15 dataset to verify the effectiveness of the proposed SPIF algorithm and iForest algorithm. The dataset we use is Data5 and the evaluation measures are area under curve (AUC) and Accuracy. AUC is an index normally used to evaluate the efficiency of classifiers, which is defined as the area under the receiver operating characteristic (ROC) curve, and Accuracy is the proportion of instances in the network traffic data that are detected correctly. The results are shown in Table 6.

As can be seen from Table 6, the AUC and Accuracy of the SPIF algorithm and iForest are basically consistent, and there is no obvious difference. In other words, the experimental results show that the proposed SPIF algorithm can effectively reduce the data processing time and improve the execution efficiency of network traffic anomaly detection without degradation in accuracy. Therefore, SPIF is suitable for anomaly detection on large-scale network traffic data.

Scalability. We compared the SPIF algorithm with HPIF algorithm and iForest algorithm based on single-machine environment to verify their scalability by investigating their running time, respectively. In this experiment, the data sample size is set to 256, and the number of iTrees is fixed at 600. The experiment results are given in Figure 5.

From Figure 5, we can see that for given fixed data sample size and the number of iTrees, the running time of iForest algorithm based on single-machine environment is increasing linearly with dataset size, whereas

Table 5. The size of different datasets.

Dataset	Data1	Data2	Data3	Data4	Data5	Data6
Size	500,000	1,000,000	1,500,000	2,000,000	2,500,000	3,000,000

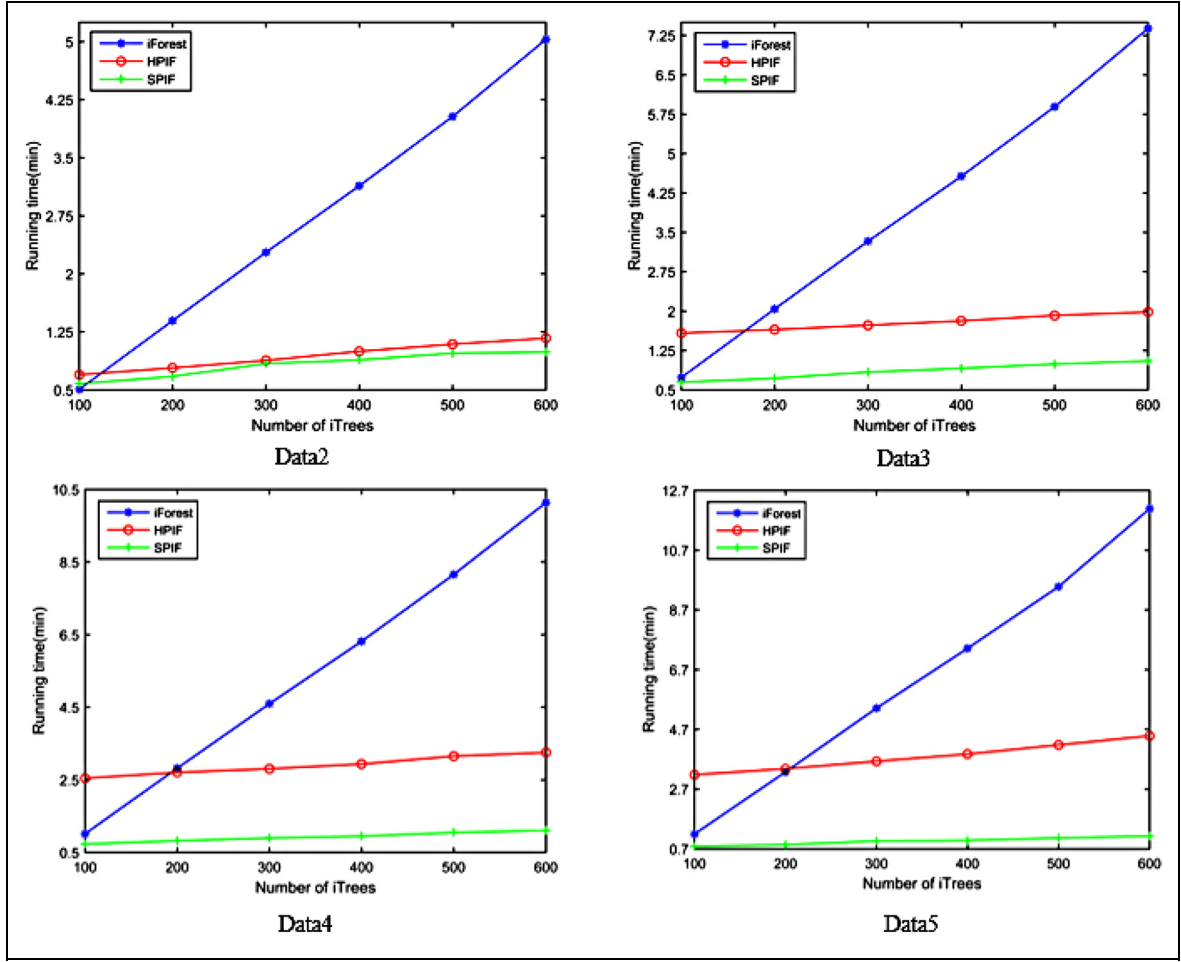


Figure 4. Running time versus different database sizes and number of iTrees.

Table 6. AUC and Accuracy of iForest and SPIF.

Algorithm	AUC	Accuracy
iForest	0.8831	86.872
SPIF	0.8927	87.144

SPIF: Isolation Forest and Spark; AUC: area under curve.

the running time of HPIF algorithm increases slowly, much lower than that of the iForest algorithm based on single-machine environment. SPIF algorithm has the shortest running time and remains stable. The experimental results show that the proposed SPIF algorithm outperforms the iForest algorithm based on single-machine environment and HPIF algorithm; hence, SPIF is more suitable for anomaly detection on large-scale network traffic data.

In order to investigate how SPIF performs when the number of computation nodes increases, we use Speedup for this purpose, which is defined as

$$Speedup = T_{iForest} / T_{SPIF} \quad (3)$$

where $T_{iForest}$ is the running time of iForest algorithm in the signal-machine environment, and T_{SPIF} is the running time of SPIF algorithm.

From Figure 6, given a fixed number of iTrees, the Speedup value of the SPIF algorithm increases gradually as the number of computation nodes increases. On the other hand, given a fixed the number of computation nodes, the Speedup of SPIF shows a steep increasing trend with the increase of the number of iTrees. The experimental results show that SPIF algorithm can effectively accelerate the process of constructing iTrees, and reduce the time consumption of anomaly evaluation. It can meet the needs of large-scale network traffic anomaly detection, and complete the process in a relatively short time. To sum up, Spark's parallel processing technology can effectively improve the efficiency of network traffic anomaly detection, and make SPIF algorithm has good extendibility.

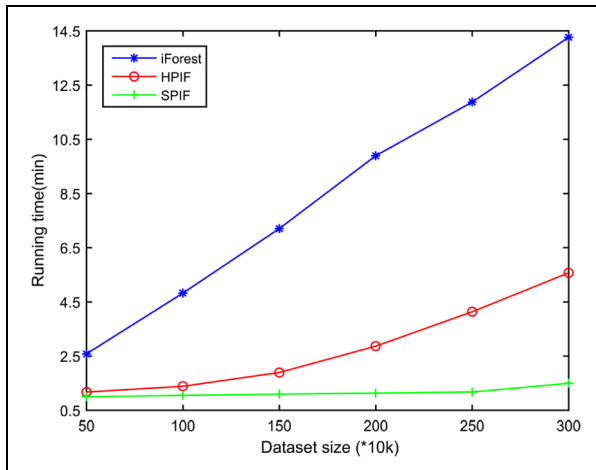


Figure 5. Running time of iForest, HPIF, and SPIF under different data sizes.

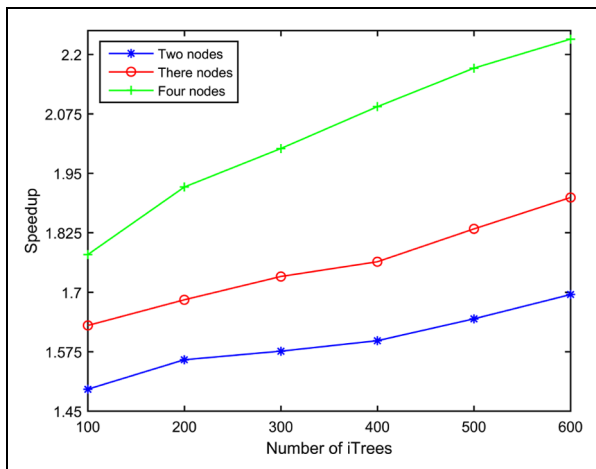


Figure 6. The Speedup of SPIF for different number of nodes and iTrees.

Conclusion

This article introduces the problem of efficient anomaly detection from large-scale network traffic data and proposes a parallel network traffic anomaly detection algorithm SPIF that is based on iForest and Spark. By taking advantages of Isolation Forest algorithm for network traffic anomaly detection and Spark technology for big data processing, SPIF can perform well in parallel the modeling process and the anomaly evaluation process. We verify the superiority of SPIF method through extensive experiments on real datasets. The experimental results show that compared with the iForest method based on single-machine environment, the SPIF algorithm not only has higher accuracy and good scalability, but also runs faster. Meanwhile, compared with the HPIF algorithm, SPIF algorithm

successfully avoids frequent disk I/O operations and significantly reduces data processing time; thus, it can be used for efficient anomaly detection from large-scale network traffic data.

In our future work, we will investigate dimension reduction techniques for our network traffic data so as to further improve the efficiency of our SPIF algorithm. And we also plan to study network security risks by analyzing anomaly detection results.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by the National Natural Science Foundation of China (no. 61363006) and the Open Projects of State Key Laboratory of Integrated Service Networks (ISN) of Xidian University (no. ISN19-13) and the National Natural Science Foundation of Guangxi (no. 2016GXNSFAA380098) and the Science and Technology Program of Guangxi (no. AB17195045).

ORCID iD

Yang Peng  <https://orcid.org/0000-0003-4043-8165>

References

1. Cai Z and Zheng X. A private and efficient mechanism for data uploading in smart cyber-physical systems. *IEEE T Netw Sci Eng*. Epub ahead of print 24 April 2018. DOI: 10.1109/TNSE.2018.2830307.
2. Cai Z, He Z, Guan X, et al. Collective data-sanitization for preventing sensitive information inference attacks in social networks. *IEEE T Depend Secure* 2018; 15: 577–590.
3. Hawkins D. *Identification of outliers (monographs on statistics and applied probability)*. 1st ed. Dordrecht: Springer, 1980.
4. Denning DE. An intrusion-detection model. *IEEE T Software Eng* 1987; 13: 222–232.
5. Paul B and Gopinathan E. Hybrid data aggregation technique in wireless sensor network through classification of fruitful messages. In: *Fourth international conference on advances in computing and communications*, Cochin, India, 27–29 August 2014, pp.157–157. New York: IEEE.
6. Abdelrahman OH, Gelenbe E, Grbil G, et al. Mobile network anomaly detection and mitigation: the NEMESYS approach. In: Gelenbe E and Lent R. (eds) *Information sciences and systems*, vol. 264. Cham: Springer, 2013, pp.429–438.
7. Salem O, Guerassimov A, Mehaoua A, et al. Sensor fault and patient anomaly detection and classification in

- medical wireless sensor networks. In: *IEEE international conference on communications (ICC)*, Budapest, 9–13 June 2013, pp.4373–4378. New York: IEEE.
8. Kadri F, Harrou F, Chaabane S, et al. Seasonal ARMA-based SPC charts for anomaly detection: application to emergency department systems. *Neurocomputing* 2016; 173: 2102–2114.
 9. Matsuda T, Morita T, Kudo T, et al. Traffic anomaly detection based on robust principal component analysis using periodic traffic behavior. *IEICE T Commun* 2017; 100: 749–761.
 10. De la Hoz E, De La Hoz E, Ortiz A, et al. PCA filtering and probabilistic SOM for network intrusion detection. *Neurocomputing* 2015; 164: 71–81.
 11. Berezinski P, Jasiul B and Szpyrka M. An entropy-based network anomaly detection method. *Entropy* 2015; 17: 2367–2408.
 12. Yu G, Hu J, Zhang C, et al. Short-term traffic flow forecasting based on Markov chain model. In: *IEEE intelligent vehicles symposium*, Columbus, OH, 9–11 June 2003, pp.208–212. New York: IEEE.
 13. Li M. Record length requirement of long-range dependent teletraffic. *Physica A* 2017; 472: 164–187.
 14. Du J, Jiang C, Qian Y, et al. Resource allocation with video traffic prediction in cloud-based space systems. *IEEE T Multimedia* 2016; 18: 820–830.
 15. Celenk M, Conley T, Graham J, et al. Anomaly prediction in network traffic using adaptive Wiener filtering and ARMA modeling. In: *IEEE international conference on systems, man and cybernetics*, Singapore, 12–15 October 2008, pp.3548–3553. New York: IEEE.
 16. Han J and Zhang JZ. Network traffic anomaly detection using weighted self-similarity based on EMD. In: *Proceedings of IEEE Southeastcon*, Jacksonville, FL, 4–7 April 2013, pp.1–5. New York: IEEE.
 17. Yu Q, Jibin L and Jiang L. An improved ARIMA-based traffic anomaly detection algorithm for wireless sensor networks. *Int J Distrib Sens N*. Epub ahead of print 18 January 2016. DOI: 10.1155/2016/9653230.
 18. Jiang D, Yao C, Xu Z, et al. Multi-scale anomaly detection for high-speed network traffic. *T Emerg Telecommun T* 2015; 26: 308–317.
 19. Huang Q and Lee PPC. LD-sketch: a distributed sketching design for accurate and scalable anomaly detection in network data streams. In: *IEEE conference on computer communications*, Toronto, ON, Canada, 27 April–2 May 2014, pp.1420–1428. New York: IEEE.
 20. Chen Z, Yeo CK, Lee BS, et al. Detection of network anomalies using Improved-MSPCA with sketches. *Comput Secur* 2017; 65: 314–328.
 21. Duong NH and Hai HD. A model for network traffic anomaly detection. In: *18th international conference on advanced communication technology (ICACT)*, Pyeongchang, South Korea, 31 January–3 February 2016, pp.644–650. New York: IEEE.
 22. Kumari R, Singh MK, Jha R, et al. Anomaly detection in network traffic using K-mean clustering. In: *3rd international conference on recent advances in information technology (RAIT)*, Dhanbad, India, 3–5 March 2016, pp.387–393. New York: IEEE.
 23. Shon T, Kim Y, Lee C, et al. A machine learning framework for network anomaly detection using SVM and GA. In: *Proceedings from the sixth annual IEEE SMC information assurance workshop*, West Point, NY, 15–17 June 2005, pp.176–183. New York: IEEE.
 24. Casas P, Fiadino P and D’Alconzo A. Machine-learning based approaches for anomaly detection and classification in cellular networks. *TMA* 2016, <http://tma.ifip.org/2016/papers/tma2016-final50.pdf>
 25. Sheikhan M and Jadidi Z. Flow-based anomaly detection in high-speed links using modified GSA-optimized neural network. *Neural Comput Appl* 2014; 24: 599–611.
 26. Liu FT, Ting KM and Zhou ZH. Isolation forest. In: *Eighth IEEE international conference on data mining*, Pisa, 15–19 December 2008, pp.413–422. New York: IEEE.
 27. Moustafa N and Slay J. The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Inform Secur J* 2016; 25: 18–31.
 28. Moustafa N and Slay J. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: *Military communications and information systems conference (MilCIS)*, Canberra, ACT, Australia, 10–12 November 2015, pp.1–6. New York: IEEE.